C++ Tutorial

First lecture

Creator

Tanvir Sorforaz

SOME MAIN DIFFERENCE BETWEEN C AND C++

Object-Oriented Programming:

- C does not support classes and objects, making it unsuitable for OOP.
- C++ introduces classes and objects, allowing for better code organization, reusability etc.

Usage:

- C is often used in systems programming, embedded systems, and for writing operating systems.
- C++ is widely used in application software development, game development, system/application software, and other domains where OOP concepts are beneficial.

Memory Management:

- C++ provides features like constructors and destructors, along with dynamic memory allocation operators new and delete, which make memory management more robust and easier compared to C.
- In C, memory management is primarily done using functions like malloc() and free().

Built-in Functions

- the C++ Standard Library provides hundreds of functions, spread across various headers like <iostream>, <string>, <vector>, <algorithm>, <map>, <set>, and more. Each header contains multiple functions, classes, and templates.
- C does not support classes and has very few built in functions.

```
#include <iostream>
using namespace std;

int main() {
  cout << "Hello World!";
  return 0;
}
```

- First test your compiler by writing these code.

- We will use VS-Code as compiler

Basic Data Types

Data Type	Size	Description
Bool (Boolean)	1 byte	Stores true or false values
char	1 byte	Stores a single character/letter/number, or ASCII values
int	2 or 4 bytes	Stores whole numbers, without decimals
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits

To know about ascii values search it on google

SIMPLE MATHEMATICS

Header file #include <math.h> / #include <cmath>

1. Addition: +
2. Subtraction: -
3. Multiplication: *
4. Division: /
5. Modulus: %
6. Exponentiation: pow(base, exponent)
7. Square root: sqrt(x)
8. Absolute value: abs(x)
9. Trigonometric functions: sin(x), cos(x), tan(x), asin(x), acos(x), atan(x)
10. Logarithm: log(x), log10(x), log2(x) Functions to calculate the natural logarithm, base 10 logarithm, and base 2 logarithm of a number, respectively.
11. Rounding functions: ceil(x), floor(x), round(x)

MINIMUM AND MAXIMUM

Header file `#include<algorithm>`

```
#include<iostream>
#include<algorithm>

using namespace std;
int main(){
    int a,b,c,d;
    cin>>a>>b>>c>>d;
    // int mn=min(a,b);      /*    2 tar jonno header file algorithm lagbe na*/
    // int mx=max(a,b);
    int mn=min({a,b,c,d});
    int mx=max({a,b,c,d});
    cout<<mn<<" "<<mx<<endl;
    return 0;
}
```

Get Relief from pain

- ```
  #include<bits/stdc++.h>
  ```
  ```

IF-ELSE

```
if (condition) {
    // Code to execute if the condition is true
} else {
    // Code to execute if the condition is false
}
```

IF-ELSE

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    int x = 10;
    if (x > 5) {
        cout << "x is greater than 5" << endl;
    }
    else {
        cout << "x is less than 5" << endl;
    }
    return 0;
}
```

IF-ELSE

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    int x = 10;
    if (x > 0) {
        cout << "x is positive" << endl;
    } else if (x < 0) {
        cout << "x is negative" << endl;
    } else {
        cout << "x is zero" << endl;
    }
    return 0;
}
```

LOOP

for loop

```
for (initialization; condition; increment/decrement) {
    // code to be executed
}
```

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    // Simple for loop to print numbers from 1 to 5
    for (int i = 1; i <= 5; ++i) {
        cout << i << " ";
    }
    cout << endl;

    return 0;
}
```

while loop

- while (condition) {
- // code to be executed
- }

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int i = 1;

    // Simple while loop to print numbers from 1 to 5
    while (i <= 5) {
        cout << i << " ";
        ++i;              // Increment the loop variable
    }
    cout << endl;

    return 0;
}
```

do-while loop

- do {
- // code to be executed
- } while (condition);

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int i = 1;

    // Simple do-while loop to print numbers from 1 to 5
    do {
        cout << i << " ";
        ++i;              // Increment the loop variable
    } while (i <= 5);
    cout << endl;

    return 0;
}
```

Difference between while and do while loop

- In do while loop body will execute once before checking condition.
- But in while loop body will never execute before checking condition.

```
int i = 0;
while (i > 0) {
    // This loop body will never execute because i is initially 0
    cout << "Inside while loop" << endl;
    i--;
}
```

```
int i = 0;
do {
    // This loop body will execute once, even though i is initially 0
    cout << "Inside do-while loop" << endl;
    i--;
} while (i > 0);
```

Some easy problems

- Write a program that calculates the sum of all even numbers between 1 and a given positive integer n.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;
    int sum = 0;
    for (int i = 1; i <= n; ++i) {

        if (i % 2 == 0) {

            sum += i;
        }
    }
    cout <<"Sum of even numbers between 1 and "<< n << " is: " << sum << endl;

    return 0;
}
```

HW : Write a program that calculates the sum of all odd numbers between 1 and a given positive integer n.

HW : Write a program that checks whether a given integer n is even or odd.

Some easy problems

- create a simple calculator using if else.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    char operation;
    double num1, num2;

    cout << "Enter an operation (+, -, *, /): ";
    cin >> operation;
    cout << "Enter two numbers: ";
    cin >> num1 >> num2;

    if (operation == '+') {
        cout << "Result: " << num1 + num2 << endl;
    } else if (operation == '-') {
        cout << "Result: " << num1 - num2 << endl;
    } else if (operation == '*') {
        cout << "Result: " << num1 * num2 << endl;
    } else if (operation == '/') {
        if (num2 != 0) {
            cout << "Result: " << num1 / num2 << endl;
        } else {
            cout << "Error: Division by zero is not allowed." << endl;
        }
    } else {
        cout << "Error: Invalid operation." << endl;
    }

    return 0;}
```